

LDPC Decoding Acceleration Architecture on Versal AI Edge for Space Platforms

Noah Perryman, Sebastian Sabogal, David Wilson, Christopher Wilson, Bert Vermeire

Voyager Technologies
Space, Defense and National Security
15378 Avenue of Science, San Diego, CA 92128
{noah.perryman, sebastian.sabogal, david.wilson,
christopher.wilson, bert.vermeire}@voyagertechnologies.com

Abstract—As new space opportunities and missions continue to grow, information mobility and communication resilience are two areas that must be evaluated for routing data through multiple space nodes in communication platforms. These next-generation systems will be comprised of both radio frequency (RF) and free-space optical (FSO) communication links, requiring satellites to provide communication services that perform demodulation and forward error correction (FEC) decoding of the received signal before re-encoding and forwarding the signal to other nodes in the network. Although this regenerative capability prevents the accumulation of errors as communications are routed throughout the network, the regenerative process incurs a significant increase in the design complexity of communication satellites, which requires both an effective FEC algorithm and high-performance processor architecture. In this paper, we propose a configurable, high-throughput, low-density parity-check (LDPC) decoder architecture accelerated on the AMD Versal Adaptive System-on-Chip (SoC), leveraging AI Engine (AIE) technology, as a potential solution for future communication networks. First, LDPC codes, a class of FEC algorithms used in advanced communications standards such as Wi-Fi (802.11n) and 10G Ethernet (IEEE802.3an), are well-suited algorithms for their high-throughput and error-correction capabilities, and the iterative and parallelizable processes in LDPC decoding are highly amenable for acceleration. Second, the Versal SoC, a next-generation space-qualified device, features an adaptive, heterogeneous architecture enabling high-performance onboard processing. When evaluated on the Versal AI Edge, our accelerator can achieve up to 4.458 Gigabits per second (Gbps) at one iteration.

TABLE OF CONTENTS

1. INTRODUCTION.....	1
2. BACKGROUND.....	2
3. LDPC DECODER ARCHITECTURE.....	4
4. RESULTS AND ANALYSIS	7
5. CONCLUSIONS.....	10
ACKNOWLEDGEMENTS	10
REFERENCES.....	10
BIOGRAPHY	12

1. INTRODUCTION

Future space communication systems, consisting of both radio frequency (RF) and free-space optical (FSO)

communication links to improve information mobility and communications resilience, are under development by many organizations [1]. However, these systems are constrained by computational needs imposed by performing forward error correction (FEC) decoding on high data-rate communication signals. One well-known class of FEC algorithms is low-density parity-check (LDPC) codes, which are highly effective algorithms due to their high-throughput and error-correction capabilities, as well as their iterative and parallelizable processes that make them well-suited for acceleration. First proposed in 1962 [2], LDPC codes were initially too resource- and compute-intensive for practical use in communication applications. However, as computing power increased, so did the viability of LDPC codes. Modern LDPC FEC decoding, hereafter referred to as LDPC decoding, provides significant error-correcting capabilities; however, it is curtailed by the amount of digital resources and power consumption it requires. Current state-of-the-art communication standards, such as Wi-Fi (802.11n) and 10G Ethernet (IEEE802.3an), use LDPC decoding but require high-performance computing capabilities to enable fast, efficient, and reliable communication.

Terrestrial high data-rate communication systems typically leverage application-specific integrated circuits (ASICs) to perform LDPC decoding [3]. However, for space systems, custom radiation-hardened ASICs are expensive and inflexible solutions. In particular, development times can be excessively long with no option for device reconfiguration to meet changing system requirements. Another alternative for LDPC decoding on space platforms is to leverage field-programmable gate arrays (FPGAs). FPGAs are highly reconfigurable and parallel architectures that can accelerate certain algorithms, especially those where parallelism can be exploited, to achieve significant performance and energy efficiency over traditional scalar processors. Unfortunately, traditional LDPC decoder implementations leveraging FPGAs are not as efficient as ASIC-based implementations [3]. Consequently, considerable FPGA resources are required for high-throughput communication architectures, making them impractical for traditional size, weight, power, and cost (SWaP-C)-constrained systems.

This challenge is exacerbated further by limitations inherent to space platforms. Commercial-off-the-shelf (COTS)

processors that are commonly and effectively used on terrestrial applications are often not suitable for space deployment [4]. While COTS processors do have the necessary processing capabilities for high data-rate communication systems, they often do not meet the strict reliability or power demands to survive or operate in the harsh radiation environment of space. Therefore, there is a significant gap in processing capability required for next-gen communication systems in future science and defense missions [5]. The AMD Versal Adaptive System-on-Chip (SoC) [6], a next-generation space-qualified processor [7], provides an adaptive, heterogeneous architecture with the necessary resources and capabilities to meet the demands of these compute-intensive communication applications.

In this research, we propose an alternative, modified min-sum LDPC decoder architecture leveraging the AI Engine (AIE) tiles of the Versal Adaptive SoC to enable high-throughput communication capabilities on space platforms. This architecture is highly configurable and scalable, with user-specified parallelization and quantization, to achieve an optimal balance between performance, power, and area tradeoffs across a wide range of mission requirements. Additionally, we explore the effects of quantization on transmitted data using Binary Phase-Shift Keying (BPSK) over the additive white Gaussian noise (AWGN) channel. When evaluated on the AMD VEK280 Versal AI Edge evaluation board, our proposed architecture achieved a throughput of up to 1.174 Gigabits per second (Gbps), 2.297 Gbps, and 4.458 Gbps at ten iterations, five iterations, and one iteration, respectively.

2. BACKGROUND

This section provides an introduction to Voyager Technologies and a cursory overview of LDPC codes with a focus on near-Earth applications. Additionally, current state-of-the-art LDPC decoder implementations are explored in this section. Furthermore, background is provided for the Versal Adaptive SoC, a promising heterogeneous-computing device for space-communication applications.

Voyager Technologies

This research was conducted by the East Coast Design Center (ECDC) within Voyager Technologies (Voyager) San Diego (formerly Space Micro Inc.), part of the Space, Defense and National Security segment. Voyager is a small, nontraditional defense contractor with innovation-driven defense technology and space solutions. Voyager's Products Business Line develops technologically advanced products and customized solutions to support the space industry. For over 20 years Voyager has designed and manufactured patented, cost-effective, radiation-tolerant/radiation-hardened space subsystems. The Voyager products ecosystem ranges, from avionics platforms to high-performance image processing computers, star trackers to rugged space cameras, from high-data-rate optical communications to frequency agile radios, and much more. Voyager products are used in a variety of applications, including, communications satellites that provides essential

communication links between people and devices on Earth and in space. The ECDC focuses on single-board computer cards, payload processors, and onboard digital solutions including SoC devices such as the AMD Versal.

LDPC Decoders

LDPC codes are a class of FEC codes that are highly efficient and widely used in modern digital communication systems [1]. They are linear block codes characterized by a sparse parity-check matrix and are represented by a bipartite or factor graph [8]. The parity-check matrix is composed of variable nodes, which represent the codeword bits (i.e., matrix columns), and check nodes, which represent the parity check equations (i.e., matrix rows). These codes can approach the Shannon limit, making them ideal for applications requiring reliable data transmission over noisy channels, including satellite communication, wireless standards, and storage systems. Typical LDPC codes are described by encoder and decoder pairs, with the encoder used to encode a signal prior to modulation and transmission, and the decoder used to decode a signal after receiving and demodulating it. LDPC codes can further be separated into binary codes and nonbinary codes [9]. The research presented in this paper focuses on the decoding of binary LDPC codes.

The LDPC decoding algorithm can be implemented using low-complexity calculations with codewords that are decoded iteratively, calculating extrinsic probability values for each bit in the transmitted message [10]. The error correction capability of these LDPC decoders is affected by the form of information provided after the demodulation process. The demodulation process can extract data from a received codeword signal in terms of log-likelihood ratios (LLRs). The sign of an LLR represents the hard decision, with a positive or negative value representing a most likely bit value of 0 or 1, respectively. The magnitude of an LLR represents the soft decision, with a greater magnitude representing higher certainty in the bit value [11].

The parity-check matrix of an LDPC decoder describes the connection between the variable nodes and the check nodes, the combination of which are used to detect errors introduced into the transmitted signal due to noise. These LDPC codes are decoded using a belief-propagation algorithm along with the parity-check matrix. Many LDPC decoding algorithms exist, but the two most common are the sum-product algorithm (SPA) [11] and the min-sum algorithm (MSA) [12]. SPA offers superior decoding performance but incurs computation complexity that requires significant computational and memory resources to implement, whereas MSA offers slightly degraded decoding performance with substantially reduced hardware and memory requirements. In this research, we focus on the MSA as a simpler approximation for energy- and resource-efficient deployment on space systems.

For the MSA-based approach, there are two main stages involved in each iteration of the decoding process [12]. At initialization, an LLR matrix is formed using the received

LLRs and the parity-check matrix. For each element of the parity-check matrix, if the element is nonzero (i.e., one), it is replaced by the received LLR corresponding to the column of the nonzero element. Afterwards, the algorithm performs an iterative process comprised of row-wise and column-wise operations.

First, the check node (CN) update, a row-wise operation, is performed. In this operation, for each row of the LLR matrix, the values and indices of the two absolute minima LLR magnitudes and the overall sign of the row are determined. Next, the nonzero elements of the row are updated. The sign of each element is updated based on its current sign and the overall sign of the row. The magnitude of each element is updated based on the index of the first minimum, where the indexed magnitude is updated to the second minimum and all other magnitudes are updated to the first minimum.

Second, the variable node (VN) update, a column-wise operation, is performed. In this operation, for each column of the LLR matrix, the nonzero elements and initial LLR values are summed. Each nonzero element is updated by the sum minus the value of the element.

During the VN update, the a posteriori LLRs of the iteration represent the vector of the column-wise sums. The vector of the signs of the a posteriori LLRs represents the a posteriori codeword, which is the final codeword after a user-specified number of iterations have elapsed. Optionally, this codeword can be checked after each iteration by performing a row-wise modulo sum for all nonzero elements of the LLR matrix. If the result is the zero vector, then the codeword is valid, and the LDPC algorithm can conclude early.

To further reduce the complexity of the MSA, the second absolute minimum for each row of the LLR matrix during the CN update can be approximated as detailed in [13]. Determining the two absolute minima increases the computational complexity of the operation that can affect throughput. To improve the efficiency of this operation, the second minimum approximation MSA (SMA-MSA) approach approximates the second minimum using two scaling factors applied to both the absolute minimum and a pseudo-second minimum.

There are many different variations of LDPC parity-check matrices used in practice, each with tradeoffs in performance, design complexity, and robustness. This research focuses on the LDPC parity-check code specified in GSFC-STD-9100 rate 7/8 [14] and CCSDS 131.0-B-3 rate 223/225 [15] standards. This parity-check matrix has a dimension of $1,022 \times 8,176$, with the following properties: $n = 8,176$, the length of the codeword, and $k = 7,154$, the length of the information sequence.

Additionally, this parity-check matrix is a member of a class of structured codes called quasi-cyclic codes, which is a matrix comprised of multiple circulants (or cyclic submatrices). For each circulant, the nonzero element shifts to the right by one element after each row, with elements at

the rightmost column wrapped to the leftmost column. Our selected LDPC parity-check matrix is a 2×16 array of 511×511 circulants. Each circulant has exactly 2 nonzero elements in each row and column, and the entire matrix has exactly 32 nonzero elements in each row and 4 nonzero elements in each column.

Current State-of-the-Art

There has been significant research into high-throughput LDPC decoder implementations by leveraging various platforms with different architectural capabilities. The most efficient designs, such as the AMD Versal RF series LDPC decoder (hard IP embedded into the Versal SoC silicon) [16], are implemented on ASICs. While able to achieve the highest throughput-performance and energy efficiency, ASIC-based implementations can be expensive to produce and have long development timelines [3]. Additionally, they are typically inflexible and are not always applicable for a broad spectrum of mission scenarios. A more exhaustive list of LDPC decoder ASICs can be found in [17]. In contrast, this research presents a highly customizable and scalable LDPC architecture within the reconfigurable elements of the Versal SoC that can be configured for different performance, power, and area tradeoffs across a wide range of mission requirements.

While the design of individual processing components for an LDPC decoder is relatively trivial, designing generically for high throughput can be considerably challenging. First, serial-based LDPC decoders are not viable due to the large size of parity-check matrices used in advanced communications standards. While each row- and column-wise operation is relatively trivial, the number of rows and columns can exceed multiple orders of magnitude. Therefore, parallel decoder implementations that exploit the inherent parallelism of the LDPC decoding algorithm are necessary to achieve the throughput requirements for high data-rate communication systems [8].

Alternative options that have been explored include traditional scalar processors, graphics processing units (GPUs), and FPGAs. COTS scalar processors, while able to achieve high degrees of parallelism, are not necessarily viable for use on space platforms. As shown in [18], scalar processors used on space platforms are significantly resource- and compute-constrained when compared to their COTS counterparts. Other options such as GPUs have been examined [19], but the feasibility of GPUs on space platforms, especially SWaP-C-constrained platforms, is not currently viable and is still being investigated.

Due to these limitations, most LDPC decoder implementations used on space platforms leverage FPGA-based implementations [3]. Current FPGA-based implementations cover a wide range of performance, resource-utilization, and efficiency capabilities and tradeoffs using different algorithms and parity-check matrices. As shown in [10], implementations that use a $1,022 \times 8,176$ parity-check matrix range in terms of throughput-

performance from 70 Megabits per second (Mbps) at 25 iterations to 11.7 Gbps at 6.8 iterations. These FPGA-based LDPC decoder architectures vary greatly and a more exhaustive comparison of FPGA-based LDPC decoder implementations can be found in [10].

Versal Adaptive SoC

The AMD Versal Adaptive SoC [6] is a heterogeneous compute platform that features a dual-core Arm Cortex-A72 application processing unit (APU), a dual-core Arm Cortex-R5F real-time processing unit (RPU), an FPGA programmable-logic (PL) fabric, AIEs, high-speed transceivers and input/output (I/O), and a Network-on-Chip (NoC). The NoC interconnect is an Advanced Microcontroller Bus Architecture (AMBA) Advanced eXtensible Interface (AXI)-interconnecting network that forms the network between the APU, RPU, FPGA, AIEs, transceivers, and I/O, and DDR memory.

Within the Versal SoC family, there are several series of devices, but in this research, we focus on the Versal AI Edge series. Within the Versal AI Edge series there are also many different devices with varied quantities of resources, packaging, and speed and temperature grades [20]. One device of interest for future space platforms is the VE2302, which is available as a space-qualified part [7]. However, due to resource and development board limitations, the research in this paper was evaluated using the VEK280 development platform, which features the VE2802. The VE2802 has considerably more resources than the VE2302 but can serve as a useful platform to demonstrate applications targeting Versal AI Edge series devices.

The Versal AI Edge series primarily features AIE-Machine Learning (AIE-ML) tiles, but, along with the Versal AI Core, Premium, and RF series devices, can also feature AIE tiles. The AIE tiles are optimized for a balance between ML and digital signal processing (DSP) application workloads, while the AIE-ML tiles are optimized for superior performance in ML application workloads. The AIE and AIE-ML tiles are a 2D mesh of processing units arranged in row-and-column format that each contain a very long instruction word (VLIW), single-instruction multiple-data (SIMD) vector processor, a reduced instruction set computer (RISC) scalar processor, and either 32 kB (AIE) or 64 kB (AIE-ML) of local data memory [21]. While the AIE and AIE-ML tiles do share some similarities, there are a few key differences. Each AIE-ML tile consists of a north-to-south or west-to-east 512-bit cascade connection between adjacent AIE-ML tiles, the direction of which is determined when the AIE-ML graph is compiled. The AIE tiles consist of a west-to-east or east-to-west 384-bit cascade interface between adjacent AIE tiles, the direction of which is determined depending on the row of the array.

Each AIE-ML tile also consists of two 256-bit buffer read inputs, one 256-bit buffer write output, one 32-bit stream input, and one 32-bit stream output. The AIE-ML array also features separate 512-kB memory tiles that can be used to

store and access data without using external memory. These memory tiles also allow predefined read and write access patterns. Further details about the AIE-ML tiles can be found in [22]. Each AIE tile consist of two 256-bit buffer read inputs, one 256-bit buffer write output, two 32-bit stream inputs, and two 32-bit stream outputs. Further details about the AIE tiles can be found in [23]. Primarily, this research focuses on the AIE-Machine Learning (AIE-ML) tiles for an LDPC decoder implementation. The architecture presented is tailored to the AIE-ML tiles and is not directly compatible with the AIE tiles, although modifications to the cascading and memory architectures could be made to make an AIE tile-based architecture.

Previous research on the Versal Adaptive SoC for space platforms targeting communication-based applications has shown promising throughput-performance and energy-efficiency results [24]. Due to these promising characteristics, we are exploring an LDPC decoder implementation on the Versal AI Edge series AIE-ML tiles. However, this architecture is significantly more complex than the previous generation of AMD FPGAs/SoCs commonly used for space (e.g., Virtex FPGA, Zynq-7000 SoC, Kintex UltraScale) and requires a unique mapping for the LDPC decoder algorithm to maximize parallelization and vectorization.

3. LDPC DECODER ARCHITECTURE

This section discusses the AIE-ML SMA-MSA LDPC decoder architecture implemented on the Versal AI Edge AIE-ML tiles, as well as provides an overview of the optimizations leveraged to increase throughput and efficiency of the design. For this research, there were several design considerations that proved to be challenging for implementing the LDPC decoder on the AIE-ML tiles. First, this section covers the design challenges presented by the distributed-memory architecture of the AIE-ML array. Next, this section covers the algorithm vectorization required to leverage the full capabilities of the AIE-ML vector processors. Furthermore, the overall tile-to-tile communication architecture is described followed by a short description of the PL-based datamover designed for this architecture. This section also covers our approach to quantization for the AIE-ML LDPC decoder. This section continues with details regarding the scalable nature of the implemented architecture and then concludes with a discussion on development and design verification for the AIE-ML tiles.

Distributed Memory Architecture

The first design challenge presented by the LDPC decoder algorithm is the size of the parity-check matrix. For our specific implementation, the 1,022×8,176 parity-check matrix requires roughly 67 Mb of memory. While streaming this data into the AIE-ML array is possible from external memory, each external memory transaction incurs significant overhead and is not viable for high-throughput implementations. Therefore, the parity-check matrix must be stored locally in AIE-ML memory. The VE2302 and VE2802

feature 85 Mb and 456 Mb of total AIE-ML memory, respectively, both of which can store the entire parity-check matrix. However, this AIE-ML memory is not directly available to every AIE-ML tile due to the distributed-memory architecture of the AIE-ML array.

Each AIE-ML tile has access to 64 kB of local data memory, as well as access to the local memories of adjacent tiles, totaling 256 kB of local memory. The 256 kB of local memory does not provide enough memory to store the parity-check matrix locally. To circumvent the storage limitations, AIE-ML tiles are individually assigned to smaller portions of the parity-check matrix. Given the 2×16 circulant-based construction of the parity-check matrix, each 511×511 circulant is assigned to one AIE-ML tile, which would reduce the storage requirements to roughly 2 Mb for each circulant, assuming 8-bit integer (INT8) data is used. While each AIE-ML tile has access to 256 kB of local memory, which would be sufficient to store a single circulant, it would require the use of every adjacent AIE-ML tile's local memory. To ensure each AIE-ML tile can store its corresponding circulant without using adjacent memories, which would still be needed to store the adjacent tile's circulant, a one-to-one mapping of circulants to AIE-ML tile memory is required. This mapping constraint makes each singular AIE-ML local memory too limited to fit an individual circulant, so further optimizations were made. The sparse nature of each circulant was leveraged along with a compressed sparse column (CSC) implementation where each column is a list of rows with nonzero values. With each row being comprised of only two ones, this results in one 2×511 matrix required for each circulant. The reduced INT8 circulant size of roughly 8 kb easily fits on each AIE-ML tile.

Algorithm Vectorization

The AIE-ML array is a distributed 2D mesh of tiles, with each tile featuring a vector and scalar processor. While the scalar processors can be leveraged to parallelize the SMA-MSA LDPC decoder, the vector processors enable parallelization and vectorization, which provides significant performance benefits over the scalar processors [22]. This approach requires the capability to read data in contiguous blocks from local AIE-ML memory, perform vector operations, and then write data in contiguous blocks to local AIE-ML memory. The limiting factors for vectorizing the LDPC decoder algorithm were the data reads from and data writes to AIE-ML memory. However, each circulant is comprised of a cyclic shift of values where the next data read or write occurs one position to the right in memory. Given the starting position of each one in the first row of the circulant, the data can then be read contiguously from memory with minimal overhead. This approach requires a 2×511 matrix to store the CSC LLR matrix. Column positions are calculated using the starting positions for each nonzero values in each row and are incremented each time a read or write occurs. This method requires two trackers, one for each nonzero value. Row positions always start in the first row and only increment to the second row once the column position overlaps with the start of the other nonzero column position.

Tile-to-Tile Communication

Following the distributed-memory architecture and algorithm-vectorization approaches, the next design challenge presented was tile-to-tile communication. Each AIE-ML tile stores the input LLR values (L_i), CSC LLR matrix (L), and column-wise sum (S). Each row-wise operation must communicate with every other successive tile in its row for the minimum calculation of the CN update and every other tile in its column for the summation calculation of the VN update. Our architecture solution is shown in Figure 1.

The AIE-ML tile operations can be separated into several different steps and into two different categories. The first category is the decoder tiles and the second is the minimum tiles. The first step involves each decoder tile reading the input LLR values from PL I/O (PLIO) over the 256-bit buffer read input, using these LLR values to initialize the local LLR matrix (L), and then storing the LLR values in local memory (L_i). Each tile then calculates the LLR matrix read indices and performs vector read operations. Absolute minima are calculated for each circulant and are then sent to the east-adjacent AIE-ML tile in the same row over the 512-bit cascade interface. Each decoder tile also reads the incoming minima from the west via the cascade interface and shifts them to the east-adjacent tile. This process is repeated until every minimum reaches the final tile in each row, which is the minimum tile. The minimum tile reads each minimum, determines the overall absolute minimum, approximates the second minimum, and then broadcasts these two minima vectors to every decoder tile in the same row. Next, each decoder tile reads this broadcast using the 32-bit input stream, calculates the LLR matrix write positions, and writes the updated minimum values to local memory. This step is repeated until each value in the LLR matrix is updated.

The final step is to read each updated value from the LLR matrix and produce the column-wise sum. To calculate the full sum, tiles in the northern row must communicate their partial sum to tiles in the southern row and vice versa. This is achieved by leveraging the remaining 256-bit buffer read and 256-bit buffer write interfaces to communicate between the northern and southern tiles. Using this sum, the LLR matrix values are updated once again.

This entire process, excluding the LLR matrix initialization, is then repeated for each iteration. During the column-wise sum step on the final iteration, each southern decoder tile uses the column sum to calculate the output message (X_t). The output message is then streamed to the PLIO using the 32-bit stream write interface.

External Memory Interface

The AIE-ML array uses two main interfaces to communicate with external interfaces: PLIO and global memory IO (GMIO). PLIO enables communication with PL-based apps and GMIO enables communication with DDR memory through the NoC interface. Each resource is limited, but PLIO interfaces are more numerous than GMIO interfaces and

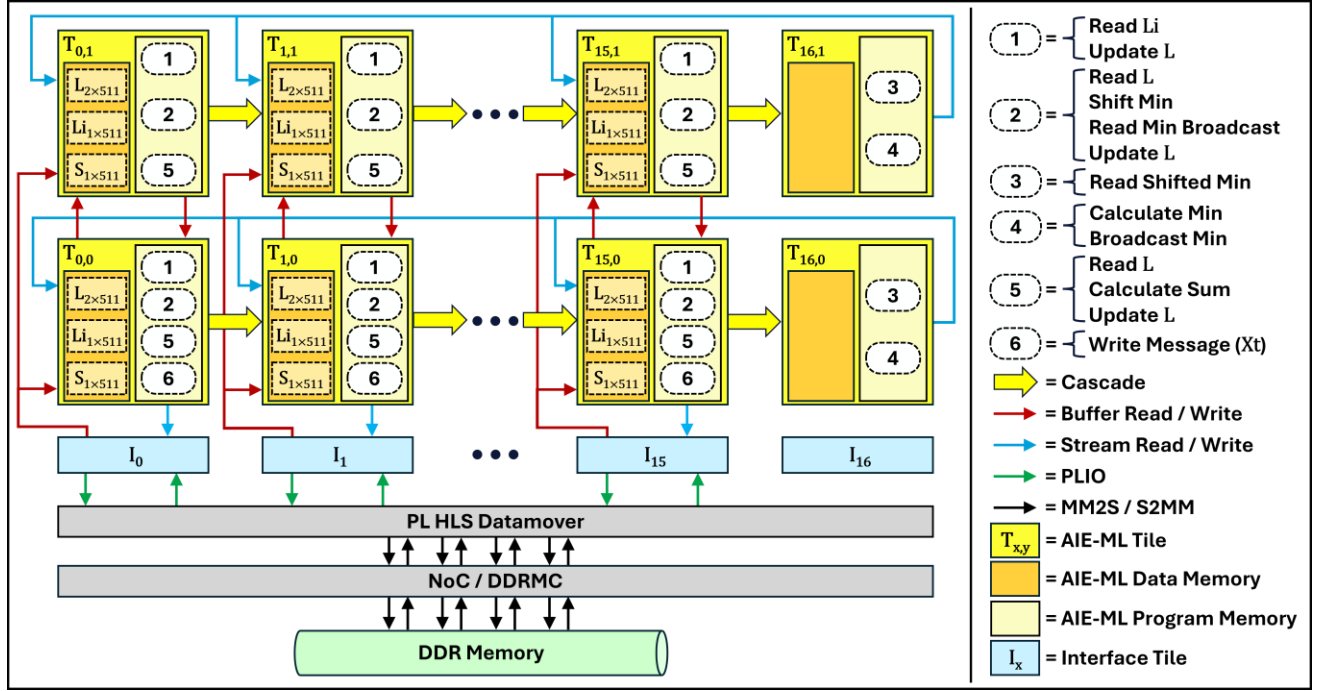


Figure 1. Proposed AIE-ML-based LDPC decoder accelerator for Versal AI Edge

provide a higher total communication bandwidth. Therefore, our LDPC decoder architecture uses PLIO to transfer data to DDR memory through a PL-based datamover generated using Vitis high-level synthesis (HLS). Additionally, there are 16 stream inputs to and 16 stream outputs from the architecture depicted in Figure 1. This I/O stream count exceeds the GMIO resources available on the VE2302 device, which limits this implementation to PLIO for the VE2302. While the VE2802 has more than enough GMIO interfaces to support this architecture, a PLIO-based architecture was used to ensure compatibility with the VE2302 device.

Data Quantization

The next design challenge for the LDPC decoder architecture involves data quantization. The AIE-ML tiles offer varying levels of performance depending on the data types used for each operation. Low-precision data types, such as 4-bit integer (INT4), INT8, and 16-bit integer (INT16) are substantially more performant and energy-efficient than single-precision floating-point (FP32); however, the approximated processing can increase the decoder susceptibilities to noise. In this research, we explored INT8, INT16, and FP32 implementations of the LDPC decoder to evaluate the tradeoffs of quantization.

An INT4 implementation was also explored; however, the required functions used in the INT8, INT16, and FP32 implementations were not fully adapted for use with INT4 data types. While these functions can be modified with additional logic, using the INT4 data type resulted in significantly increased AIE-ML program memory that surpassed the 16-kB limit. This required the removal of

compiler optimizations to meet the 16-kB program memory requirement, which significantly decreased throughput-performance. Significant overhead that could not be attributed to what has already been discussed was also observed. Future releases of the AMD Vitis tools may help streamline this process, but inevitably we did not find the INT4 implementation viable for this research due to the above observations.

Development and Design Verification

The last design challenge to discuss is development and design verification for the AIE-ML tiles. While there were many different steps that required verification during the design process, they can be grouped into three major categories. The first category deals with verification of the AIE-ML adaptive data flow (ADF) graph specification. The second category covers verification of the PL datamover IP and AIE-ML graph kernel code using the AIE multi-threaded simulator (aiesimulator). The last category covers system-level verification using single-threaded baremetal (i.e., without an operating system) and using AMD's embedded Linux solution, PetaLinux. The design verification methods discussed in this paper are not meant to represent an exhaustive list of AIE verification methods, but rather a small subset of techniques that we found useful. There are many other design verification methods available that leverage the Vitis tools and AIE debugging techniques.

Once the general architecture for the LDPC decoder was created, it needed to be mapped to the AIE-ML tiles using an ADF graph specification. The streaming connections between tiles, PL, local memory, and external memory are configured during this process. Design verification for this

step was simpler than subsequent design verification steps. For the ADF graph verification, built-in Vitis tools were leveraged to visualize kernel and memory placement for each tile along with stream connections to other tiles, PL, local memory, and external memory. Using this visual verification process enabled us to quickly verify that the LDPC decoder architecture was mapped correctly to the AIE-ML tiles.

The next verification category covers the verification of the PL datamover IP and AIE-ML graph kernel code using the aiesimulator. The PL datamover IP was verified using Vitis HLS testbenches by checking that data being read by the memory-mapped to stream (MM2S) IP matched data being written by the stream to memory-mapped (S2MM) IP in a loopback configuration. The aiesimulator is a cycle-approximate simulator that calculates core vector load and memory accesses precisely while also models tile-to-tile communication and external PL and DDR memory interfaces. Whereas the ADF graph specifies graph connectivity, the graph kernel code specifies the fundamental building blocks that determine AIE-ML tile operation and how the ADG graph interfaces are used. The kernel code covers the individual functions that each tile performs, labeled as items 1-6 in Figure 1. The verification process included reading and storing the input LLR values (L_i) appropriately, reading from and writing to the LLR matrix (L), calculating, shifting, and broadcasting the minima, and summing the LLR matrix (L) columns. Throughout each of these verification steps, simulator output was recorded along with special AIE print statements to determine vector register contents.

While each kernel function was unique, each function was evaluated using a similar verification process. The first step was isolating each function. For each individual function, a simple kernel, or group of kernels if tile-to-tile communication was necessary, was created to ensure that predetermined data could be input into the kernel and verified at the output and at intermediate steps of the kernel. The intermediate values within the vector registers could be verified using the AIE-ML print statements while the output values were stored in external memory over a simulated PLIO connection. These values were then compared against golden values to ensure correctness. Verification steps were performed individually and then incrementally to ensure correct operation. Once the functions of a kernel were verified, they were adapted to every other AIE-ML tile to evaluate the entire AIE-ML array. Afterwards, the entire system was verified using an LDPC code with predefined noise to ensure that the message was correctly decoded.

Once the LDPC decoder architecture was verified, the last step was to verify the APU controller at the system level on hardware. The system was first verified using baremetal code developed in Vitis, with the final verification being performed using PetaLinux. The baremetal approach was

performed first due to its rapid prototyping capabilities and low-level functionality, which allowed the system to be verified quickly. Both methods followed a similar process in which baremetal code was easily adapted to PetaLinux using the Xilinx Runtime (XRT) application programming interface (API). Given that the LDPC decoder architecture was verified at multiple levels using the aiesimulator, system-level verification relied solely on using an LDPC code with predefined noise to ensure that the message was decoded successfully. This approach also allowed for testing of other parameters more easily than on the aiesimulator such as the number of iterations for the LDPC code and the number of LDPC codewords to decode. The output for every parameter change at the system level on hardware was compared against golden data generated in software.

4. RESULTS AND ANALYSIS

This section provides an overview of the performance evaluation of the LDPC decoder architectures on the Versal AI Edge device. First, this section highlights the bit error probability performance for the INT8, INT16, and FP32 LDPC decoder implementations. Next, the resource utilization for the LDPC decoder architecture, as well as the design scalability are explored. This section concludes with an analysis of the throughput-performance, power consumption, and energy efficiency of the LDPC decoder architectures.

Bit Error Probability Performance

To fully characterize the tradeoffs of the INT8, INT16, and FP32 implementations for our LDPC decoder architecture, we calculated various bit error rates (BERs) for each data type using a BPSK modulation scheme over the AWGN channel. This bit error probability performance details the performance of each data type at varied signal-to-noise ratios (SNRs) and includes the theoretical performance for uncoded BPSK as a baseline. The SNR is a ratio that measures the energy per bit (E_b) to the noise power spectral density (N_0).

As shown in Figure 2, each implementation had decreased bit error probabilities when compared to uncoded BPSK for SNRs over 3 dB, showcasing the reliability improvements even just one iteration of the LDPC decoder can provide for certain environments. As expected, increasing the number of LDPC iterations significantly reduced the bit error probability for similar SNRs for each data type. Additionally, INT16 and FP32 data types resulted in similar bit error probabilities for the same SNRs across varying iterations for our architecture, as shown by the groupings in Figure 2.

This result indicates that reducing the data precision down to INT16 does not incur any significant increases in the bit error probability. However, significant increases in the bit error probability were seen for each INT8 implementation when compared to INT16 and FP32 implementations. This outcome is most likely due to data overflow occurring during the column sum operations.

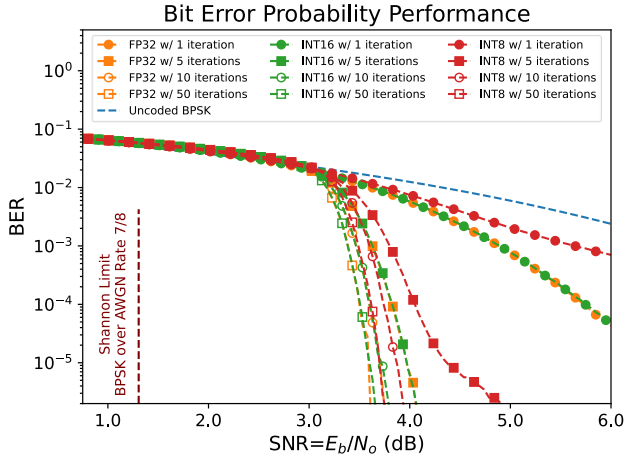


Figure 2. LDPC decoder bit error probability performance

Resource Utilization

Our LDPC decoder architecture was designed to be scalable, with up to seven decoder compute units (CUs) that can be instantiated in parallel on the AIE-ML tiles. Regardless of the data precision, each LDPC decoder implementation leverages the same base LDPC decoder architecture as shown in Figure 1. The resource utilization for this architecture with one (CU1), three (CU3), and seven (CU7) LDPC CUs in parallel is shown in Table 1, with each implementation sharing the same number of compute units utilizing the same total number of PL and AIE-ML resources. Resource utilization was collected from post-implementation utilization reports using the AMD Vivado and Vitis design tools.

The trend for resource utilization has a direct relationship to the number of compute units used. Each CU uses 34 AIE-ML tiles, resulting in 102 and 238 AIE-ML tiles used for the CU3 and CU7 architectures, respectively. The major PL resources (i.e., look up tables (LUTs), flip flops (FFs), block RAM (BRAM) blocks, and DSP slices) follow the same trend, with the CU3 and CU7 implementations utilizing approximately 3 $\times$ and 7 $\times$ the number of resources as the CU1 implementation, respectively. Not only does this scalability offer increased throughput-performance, but previous research has demonstrated that leveraging multiple CUs in parallel, configured for coarse-grained triple-modular

redundancy, can provide significant reliability improvements in exchange for reduced throughput [25]. While not shown in Table 1, the CU1 implementation was also designed to be supported by the VE2302 device. Implementations with more than one CU in parallel are not supported by the VE2302.

Throughput Performance

The next important metric to characterize is the throughput performance of the LDPC decoder architecture for each data type and CU implementation. Performance was measured with the AIE-ML tiles and the PL datamover operating at 1,250 MHz and 400 MHz, respectively. Figure 3 shows the throughput for each data type for varied iteration counts. First, these results highlight the inverse relationship between iteration count and throughput. While reliability and the likelihood of correctly decoding a message generally increases with the iteration count, the throughput decreases. Increasing the iteration count from one to five and one to ten resulted in average reductions in throughput of 3.605 $\times$ and 7.138 $\times$, respectively.

Overall, our LDPC decoder was able to achieve a throughput of up to 4,458 Mbps for one iteration, 2,297 Mbps for five iterations, and 1,174 Mbps for ten iterations. As expected, decreasing the data precision to INT8 resulted in the highest throughput LDPC decoder architecture, followed by the INT16 and FP32 architectures. On average, the INT8 LDPC decoder architecture resulted in 1.730 $\times$ and 3.841 $\times$ speedups when compared to the INT16 and FP32 LDPC decoder architectures, respectively.

Additionally, increasing the number of compute units in parallel increased the throughput. Increasing the CUs from one to three resulted in a 2.889 $\times$ average increase in throughput and increasing from one to seven resulted in a 5.580 $\times$ average increase in throughput across varying number of iterations. Using five and ten iterations for each data type increased the performance by an almost equal amount to the increase in the number of CUs used, but this trend was not observed for one iteration. Across each data type for one iteration, performance began to be limited and did not follow a linear trend. For one iteration, increasing from one CU to seven CUs only resulted in a 3.615 $\times$ average increase in throughput, which indicates the presence of a limiting factor. Since this limiting factor was not observed across varied

Table 1. LDPC decoder resource utilization

Resource	Available VE2802 Resources	CU1 Utilization		CU3 Utilization		CU7 Utilization	
		Total	%	Total	%	Total	%
LUTs	520,704	24,305	4.668	76,576	14.71	214,549	41.20
FFs	1,041,408	32,435	3.115	104,703	10.05	307,615	29.54
BRAM Blocks	600	16	2.667	48	8.000	112	18.67
DSP Slices	1,312	16	1.220	48	3.659	112	8.537
AIE-ML Tiles	304	34	11.18	102	33.55	238	78.29

iterations, we speculate that these diminishing returns are due to limitations in memory bandwidth.

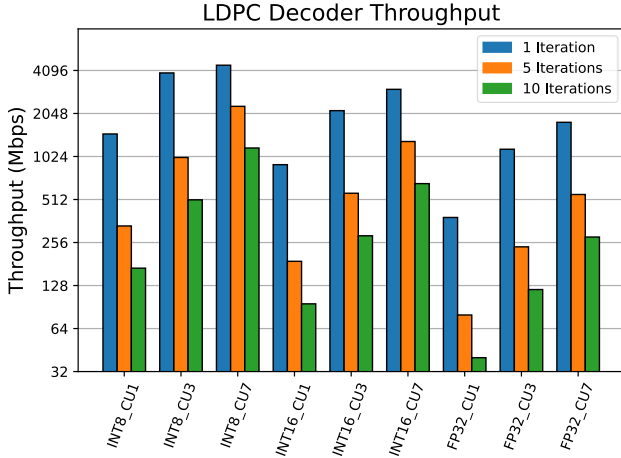


Figure 3. LDPC decoder throughput performance

While this LDPC decoder application does provide a unique architecture implementation, especially for space platforms, it is important to discuss the limitations of this architecture, which can be separated into three categories: vector read and write operations, tile-to-tile communication operations, and vector arithmetic operations.

First, the vector read and write operations do incur some overhead due to data alignment issues. AIE-ML applications can load from data memory into vector registers for vector operations, but this data must follow strict alignment requirements. For AIE-ML applications, data must be aligned on 256 bits. While the buffer read and write operations were able to be vectorized by exploiting the cyclic nature of each circulant, the starting positions of each nonzero value are not necessarily aligned on 256-bit memory addresses. Reading from and writing to data memory that is not 256-bit aligned incurs an unavoidable overhead. Additionally, each circulant submatrix of the LLR matrix has a column width of 511, which is also not 256-bit aligned. To alleviate this issue, each circulant is padded with a zero column to ensure that this alignment requirement is met.

Next, the tile-to-tile communication operations demonstrate a unique balance between exploiting parallelism in the LDPC decoder architecture and communication latency between each AIE-ML tile. Each communication interface between AIE-ML tiles requires each tile to be synchronized. Due to this synchronization requirement, AIE-ML tiles will stall when communicating with adjacent AIE-ML tiles until the adjacent AIE-ML tile is ready to communicate, which will decrease overall throughput. Additionally, every AIE-ML decoder tile needs to communicate with every successive decoder tile in its row during the minimum shift operation. While these shift operations leverage the 512-bit cascade interface, the minimum tiles must broadcast their final values using the significantly slower 32-bit stream interface, which incurs additional overhead each time the signal is broadcast.

The last major source of tile-to-tile communication overhead involves the partial-sum communication between the top and bottom row of decoder tiles. Due to the limited remaining communication interfaces available to each AIE-ML tile, the partial-sum communication is performed over the 256-bit buffer read and write interfaces. These interfaces are typically synchronous and only send or receive data once per kernel execution. However, our approach requires reads and writes every LDPC decoder iteration, which occurs multiple times per kernel execution. To resolve this synchronization, the interfaces were used asynchronously, where synchronization is manually controlled by interface locks in the kernel. This approach incurs additional latency each time these locks are acquired and released, which occurs for each LDPC iteration.

Power Consumption and Energy Efficiency

Unlike the relationship between performance and data precision, there were minimal changes in power consumption between INT8, INT16, and FP32 LDPC decoder implementations with the same number of CUs, as shown in Figure 4. Each implementation was tested with one (L1), five (L5), and ten (L10) LDPC iterations. Power was measured using the Board Evaluation and Management tool on the VEK280 system controller, which can monitor the current and voltage of power rails for the development board. The average power consumption for each implementation varied by only milliwatts. Increasing the number of CUs did see significant increases in power consumption though, with a maximum average power consumption of 21.38 W for the CU7 architecture. On average, increasing from one to three CUs resulted in an increase of 1.053 W while increasing from one to seven CUs resulted in an increase of 7.559 W. As discussed previously, unlike FPGA-based implementations, leveraging lower-precision data does not result in a lower resource utilization for the AIE-ML tiles. Each implementation leverages the same architecture, which indicates that for our architecture, quantization does not provide any significant reductions in power consumption.

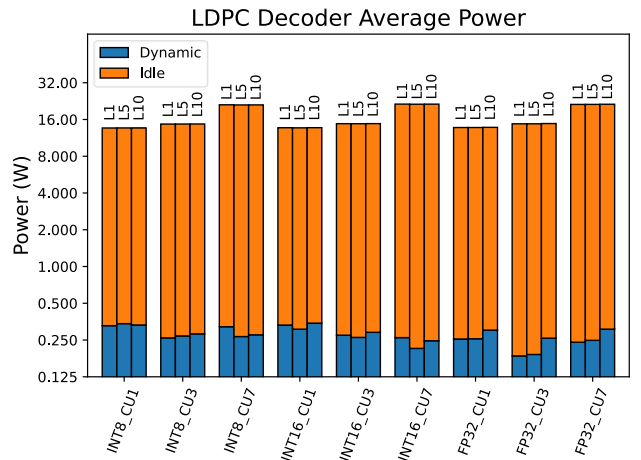


Figure 4. LDPC decoder average power consumption

Due to the similar resource utilization and power consumption results across different data types, the LDPC

decoder energy efficiency, given by Figure 5, follows a similar trend as the throughput, but with some exceptions. The INT8 LDPC decoder implementation, achieving a maximum of 268.2 Mbps/W, was the most energy efficient for each combination of CUs, followed by the INT16 and FP32 implementations, respectively. On average, increasing the number of CUs from one to three resulted in a $2.683\times$ increase in energy efficiency while increasing from one to seven resulted in a $3.598\times$ increase in energy efficiency.

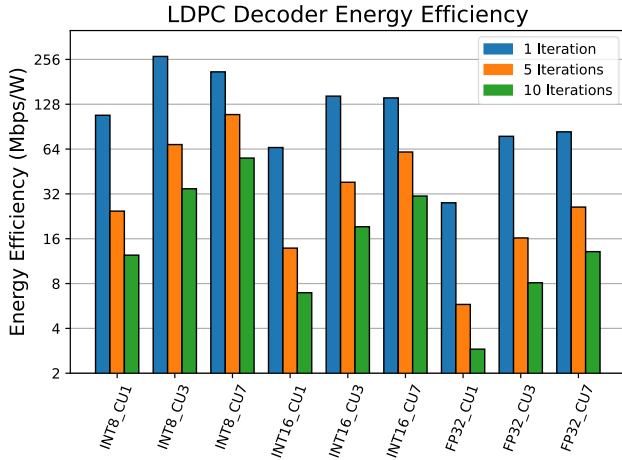


Figure 5. LDPC decoder energy efficiency

The exception here however is due to the memory bandwidth limitation discussed in the previous section. Due to this limitation, there is a $1.270\times$ and $1.027\times$ reduction in energy efficiency for the INT8 and INT16 implementations, respectively, when increasing from three to seven CUs. The FP32 implementation is not affected by this reduction in energy efficiency. This observation indicates that, due to the memory bandwidth limitation, while increasing the number of CUs in parallel improves performance, it does not necessarily improve energy efficiency for our architecture.

A similar trend was observed for the number of iterations, with an inverse relationship between the iteration count and energy efficiency. Implementations with more iterations are less energy efficient. On average, increasing the number of iterations from one to five and one to ten resulted in a $3.603\times$ and $7.148\times$ reduction in energy efficiency, respectively.

5. CONCLUSIONS

There is an increasing need in future science and defense missions for communication systems requiring high-performance capabilities for computationally intensive advanced applications. This challenge is exemplified with low-density parity-check (LDPC) decoding, which provides a high data-rate solution for efficient and robust communications in space applications but correspondingly requires significant power and processing resources. In this research, we demonstrate the efficacy of an LDPC decoder acceleration architecture for the AMD Versal Adaptive System-on-Chip (SoC) platform by leveraging its AI Engine (AIE) technology. The acceleration of LDPC decoding on

platforms such as the Versal Adaptive SoC can substantially reduce the overhead associated with regenerative communications. Our LDPC decoder architecture explores a novel approach in partitioning and vectorizing LDPC functions across multiple AIE tiles for parallel processing. Furthermore, this architecture was designed for scalability, where the parallelization and quantization can be configured to accommodate a wide range of mission requirements. The architecture supports up to seven parallel LDPC decoder compute units (CUs) and data types including 8-bit integer (INT8), 16-bit integer (INT16), and single-precision floating-point (FP32). In our evaluation, the LDPC decoder architecture was implemented with one, three, and seven CUs in parallel on the AMD VEK280 platform and achieved a throughput of up to 4.458 Gbps, 2.297 Gbps, and 1.174 Gbps for one iteration, five iterations, and ten iterations, respectively. This approach enables the design to be modified to allow for more CUs in parallel, drastically increasing throughput. Additionally, our architecture demonstrated that INT16 quantization incurred a minimal increase in bit error rate (BER) over similar signal-to-noise (SNR) ratios when compared to FP32 implementations. However, we observed significant increases in BER for the INT8 quantization. This introduces a significant tradeoff as the INT8 implementations showed substantial increases in throughput performance and energy efficiency over INT16 and FP32 implementations. While this architecture demonstrates significant performance capabilities, there are limitations that restrict the achievable performance. Mainly, the distributed-memory architecture of the AIE-ML tiles and latency incurred from tile-to-tile communication and synchronization limits the throughput of the LDPC decoder on the Versal AIE tiles. Despite these limitations, these results highlight the capabilities of the Versal Adaptive SoC platform to meet the demand for compute-intensive communication systems requiring high data-rate solutions for efficient and robust communications in space applications.

ACKNOWLEDGEMENTS

This research was funded with the support of the United States Space Force (USSF) Small Business Innovation Research (SBIR) contract F2D-7189. The authors also recognize initial investigations and contributions conducted by Justin Killam and Saeed Nejatali at Voyager Technologies San Diego.

REFERENCES

- [1] S. Weston, C. Burkhard, J. Stuple, R. Ticknor, B. Yost, R. Austin, P. Galchenko, L. Newman, and L. Soto, "State-of-the-Art Small Spacecraft Technology," NASA TP-20250000142, February 2025.
- [2] R. Gallager, "Low-density parity-check codes," in *IRE Transactions on Information Theory*, vol. 8, no. 1, pp. 21-28, January 1962, doi: 10.1109/TIT.1962.1057683.
- [3] O. Ferraz et al., "A Survey on High-Throughput Non-Binary LDPC Decoders: ASIC, FPGA, and GPU Architectures," in *IEEE Communications Surveys &*

- Tutorials*, vol. 24, no. 1, pp. 524-556, Firstquarter 2022, doi: 10.1109/COMST.2021.3126127.
- [4] A. D. George and C. M. Wilson, "Onboard Processing With Hybrid and Reconfigurable Computing on Small Satellites," in *Proceedings of the IEEE*, vol. 106, no. 3, pp. 458-470, March 2018, doi: 10.1109/JPROC.2018.2802438.
 - [5] D. Miranda, "2020 NASA Technology Taxonomy," NASA HQ-E-DAA-TN76545, January 2020.
 - [6] AMD-Xilinx. Versal: The First Adaptive Compute Acceleration Platform (ACAP) White Paper WP505. Technical report, AMD-Xilinx, September 2020.
 - [7] AMD-Xilinx. Radiation Tolerant Versal AI Edge Series Data Sheet DS955. Technical report, AMD-Xilinx, January 2025.
 - [8] M. M. Mansour and N. R. Shanbhag, "High-throughput LDPC decoders," in *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 11, no. 6, pp. 976-996, Dec. 2003, doi: 10.1109/TVLSI.2003.817545.
 - [9] S. El Hassani, M. H. Hamon and P. Pénard, "A comparison study of binary and non-binary LDPC codes decoding," *SoftCOM 2010, 18th International Conference on Software, Telecommunications and Computer Networks*, Split, Croatia, 2010, pp. 355-359.
 - [10] P. Hailes, L. Xu, R. G. Maunder, B. M. Al-Hashimi and L. Hanzo, "A Survey of FPGA-Based LDPC Decoders," in *IEEE Communications Surveys & Tutorials*, vol. 18, no. 2, pp. 1098-1122, Second quarter 2016, doi: 10.1109/COMST.2015.2510381.
 - [11] F. R. Kschischang, B. J. Frey, and H.A. Loeliger, "Factor graphs and the sum-product algorithm," *IEEE Trans. Information Theory*, vol. 47, no. 2, pp. 498-519, 2001, doi: 10.1109/18.910572.
 - [12] F. Angarita, J. Valls, V. Almenar, and V. Torres, "Reduced-complexity min-sum algorithm for decoding LDPC codes with low error-floor," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 61, no. 7, pp. 2150-2158, Jul. 2014, doi: 10.1109/TCSI.2014.2304660.
 - [13] J. M. Català-Pérez, J. O. Lacruz, F. García-Herrero, J. Valls, and D. Declercq, "Second Minimum Approximation for Min-Sum Decoders Suitable for High-Rate LDPC Codes," in *Circuits, Systems, and Signal Processing*, vol. 38, no. 11, pp. 5068-5080, April 2019, doi: 10.1007/s00034-019-01107-z.
 - [14] CCSDS recommendation 131.0-B-2, TM Synchronization and Channel Coding. Blue Book. August 2011 (or later version), <http://www.ccsds.org>.
 - [15] Low Density Parity Check Code for Rate 7/8. Technical Report. GSFC-STD-9100, March 2017.
 - [16] AMD-Xilinx. AMD Versal RF Series for Advancing Electromagnetic Spectrum Operations Solution Brief. Technical report, AMD-Xilinx, 2024.
 - [17] S. Shao et al., "Survey of Turbo, LDPC, and Polar Decoder ASIC Implementations," in *IEEE Communications Surveys & Tutorials*, vol. 21, no. 3, pp. 2309-2333, thirdquarter 2019, doi: 10.1109/COMST.2019.2893851
 - [18] T. M. Lovelley and A. D. George, "Comparative Analysis of Present and Future Space-Grade Processors with Device Metrics," *Journal of Aerospace Information Systems*, vol. 14, no. 3, pp. 184-197, Mar. 2017, doi: 10.2514/1.i010472.
 - [19] Z. Liu and L. Zhao, "High-throughput software LDPC decoder on GPU," in *EURASIP Journal on Advances in Signal Processing*, vol. 2024, no. 91, October 2024, doi: 10.1186/s13634-024-01190-8.
 - [20] AMD-Xilinx. AMD Versal AI Edge Series Product Selection Guide XMP464. Technical report, AMD-Xilinx, June 2025.
 - [21] AMD-Xilinx. AMD AI Engine Technology. Technical report, AMD-Xilinx, 2025.
 - [22] AMD-Xilinx. Versal Adaptive SoC AIE-ML Architecture Manual AM020. Technical report, AMD-Xilinx, June 2025.
 - [23] AMD-Xilinx. Versal Adaptive SoC AI Engine Architecture Manual AM009. Technical report, AMD-Xilinx, August 2025.
 - [24] N. Perryman, J. Goodwill, S. Sabogal, D. Wilson, C. Wilson, and A. George, "Comparative Analysis of Next-Generation Space Computing Applications on AMD-Xilinx Versal Architecture," *Journal of Aerospace Information Systems*, vol. 22, no. 2, pp. 103-115, Feb. 2025, doi: 10.2514/1.I011455.
 - [25] N. Perryman, S. Sabogal, C. Wilson and A. George, "Dependable DPU Architectures on AMD-Xilinx Versal Adaptive SoCs for Space Applications," in *IEEE Transactions on Aerospace and Electronic Systems*, vol. 61, no. 3, pp. 6629-6646, June 2025, doi: 10.1109/TAES.2025.3527938.

BIOGRAPHY



Noah Perryman is a Senior Electrical Design Engineer at Voyager Technologies. He received his Ph.D. in Electrical and Computer Engineering from the University of Pittsburgh for strategies to enable reliable next-generation applications on embedded space platforms. His research focuses on the evaluation, hardware, FPGA, and software design of next-generation embedded platforms for space systems.



Sebastian Sabogal is a Senior FPGA Engineer at Voyager Technologies. He received his Ph.D. in Electrical and Computer Engineering from the University of Pittsburgh for selective and adaptive resilience strategies in reconfigurable space systems and applications. His research interests include reconfigurable and fault-tolerant computing.



David T. Wilson is a Senior FPGA Engineer at Voyager Technologies. He received his Ph.D. in Electrical and Computer Engineering and his Master of Science in Electrical and Computer Engineering from the University of Florida. His research interests include reconfigurable computing, FPGA overlays, and high-level synthesis.



Christopher M. Wilson is the Product and Technology Officer at Voyager Technologies and currently serves as head of the East Coast Design Center (ECDC). He received his Ph.D. in Computer Engineering from the University of Florida for fault-tolerant mitigation schemes on hybrid space devices. His research interests include fault-tolerant techniques on hybrid architectures and radiation effects on commercial devices.



Bert Vermeire is the Chief Technology Officer and joined Space Micro (acquired by Voyager Technologies) from Arizona State University, where he was associate professor of research and director of the Wireless and Nanotechnology research programs. He has been the principal investigator on a wide variety of advanced communications technology programs, including optical communications, Software Defined Radios, radiation effects on microcircuits and novel design techniques for mixed signal circuits. His recent projects include development of the Mustang Space Relay Optical Communications Terminal, a software defined transponders and transmitters that flew on NASA's Interface Region Imaging Spectrograph (IRIS) and Lunar Atmospheric and Dust Environment Explorer (LADEE) missions. He holds 11 US patents and has authored more than 60 peer-reviewed publications.